204252 Data Structures and Analysis

Chapter 9: Sets, Maps, and Dictionaries

Thapanapong Rukkanchanunt and Jakarin Chawachat

August 10, 2026

Computer Science Department

Outline

The Set ADT

The Map ADT

The Dictionary ADT

Summary of Implementations

The Set ADT

The Set ADT Definition & Properties

A **Set** is a collection of distinct objects with no duplicates and no explicit notion of keys or order.

Properties of a Set in C++ (`std::set`)

- **Uniqueness**: All elements are unique. Duplicate insertions are ignored.
- **Sorted**: Elements are maintained in a sorted order (increasing by default).
- **Immutable**: Elements cannot be modified directly in place (you must erase and re-insert).
- **Under the Hood**: Typically implemented using a balanced binary search tree (Red-Black Tree).

Examples

- **Databases**: Set of records in a table; set intersections for conjunction queries.
- **Search Engines**: Set of URLs matching a specific keyword.
- **Real World**: Email addresses, Facebook friends, store inventory items.

Fundamental Functions of Set ADT

Core ADT Functions:

- `insert(e)`: Add element e. Returns iterator to its location (fails if duplicate).
- `find(e)`: Returns iterator to e, or `end()` if not found.
- `erase(e)`: Removes element e.
- `begin()` / `end()`: Iterators to access set elements.

STL Set Extensions:

- `lower_bound(e)`: Returns iterator to first element $\geq e$.
- `upper_bound(e)`: Returns iterator to first element $> e$.
- `equal_range(e)`: Returns iterator pair representing range of elements equal to e.

C++ STL Set: Insertion Details

The STL `std::set` provides two main insertion signatures:

1. One-parameter insert

```
pair<iterator, bool> insert(const Object & x);
```

- Returns a pair containing the iterator to the element (newly inserted or existing) and a boolean indicating whether insertion succeeded.

C++ STL Set: Insertion Details

2. Two-parameter insert (with hint)

```
iterator insert(iterator hint, const Object & x);
```

- `hint` points to the position where `x` should go.
- If the hint is accurate, insertion takes $\mathcal{O}(1)$ time. If not, it falls back to standard $\mathcal{O}(\log N)$ insertion.

Example: Hint Insertion Optimization

```
set<int> s;
for (int i = 0; i < 100000; ++i) {
    s.insert(s.end(), i); // O(1) time because s.end() is always the correct
        position!
}
```

C++ STL Set: Erase and Find Details

Three Versions of `erase`

1. **Erase by value**: `int erase(const Object & x)`
   - Removes `x` if present. Returns number of items removed (0 or 1).
2. **Erase by iterator**: `iterator erase(iterator itr)`
   - Removes object at `itr`. Returns iterator to the following element. `itr` becomes invalid/stale!
3. **Erase range**: `iterator erase(iterator start, iterator end)`
   - Removes all elements in `[start, end)`.

C++ STL Set: Erase and Find Details

The `find` Member Function

```
iterator find(const Object & x) const;
```

- Returns an iterator to the element if found, or `end()` if not.
- Returning an iterator is more informative than a simple boolean because it allows direct access to the found item's location without re-searching.

C++ STL Set: Custom Ordering

By default, elements are ordered using `std::less<Object>`, which invokes the < operator. We can pass a custom comparison function object to change this:

```
struct CaseInsensitiveCompare {
    bool operator()(const string& lhs, const string& rhs) const {
        return strcasecmp(lhs.c_str(), rhs.c_str()) < 0;
    }
};

int main() {
    set<string, CaseInsensitiveCompare> s;
    s.insert("Hello");
    s.insert("HeLLo"); // Duplicate case-insensitive string! Ignored.
    cout << "Size: " << s.size() << endl; // Prints 1
}
```

Initializing and Traversing a Set

```cpp
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s1; // Default: Ascending Order
    set<int, greater<int>> s2; // Descending Order
    set<int, greater<int>> s3 = {6, 10, 5, 1}; // s3 = {10, 6, 5, 1}
    set<int, greater<int>> s4(s3); // Copy constructor

    int arr[] = {10, 4, 5, 61};
    set<int> s5(arr, arr + 4); // s5 = {4, 5, 10, 61}

    // Traversal using iterators
    for (set<int>::iterator it = s5.begin(); it != s5.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;
}
```

Set Operations: Mathematical Definitions

For two sets A and B, we define:

Set A	Set B	Union ($A \cup B$)	Intersection ($A \cap B$)	Difference ($A \setminus B$)
{32, 20, 12, 6}	{42, 12, 6}	{32, 20, 12, 6, 42}	{12, 6}	{32, 20}
{21, 1, 70, 9}	{}	{21, 1, 70, 9}	{}	{21, 1, 70, 9}
{5, 27, 18}	{27}	{5, 27, 18}	{27}	{5, 18}
{27}	{5, 27, 18}	{5, 27, 18}	{27}	{}

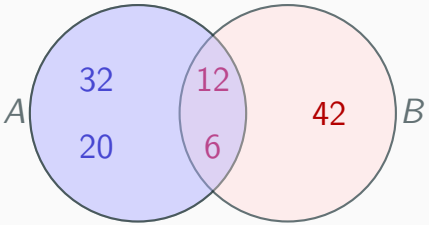
- **Union**: Contains all elements from both sets.
- **Intersection**: Contains elements that belong to both sets.
- **Difference**: Contains elements in A that are not in B.

Implementing Set Union & Intersection

```cpp
template <typename T>// Generic Template Set Union
set<T> getUnion(set<T> a, set<T> b) {
    set<T> result = a;
    for (typename set<T>::iterator it = b.begin(); it != b.end(); ++it) {
        result.insert(*it);
    }
    return result;
}
template <typename T>// Generic Template Set Intersection
set<T> getIntersection(set<T> a, set<T> b) {
    set<T> result;
    for (typename set<T>::iterator it = a.begin(); it != a.end(); ++it) {
        if (b.find(*it) != b.end()) {
            result.insert(*it);
        }
    }
    return result;
}
```

Set Union Trace: Step-by-Step

Union of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.



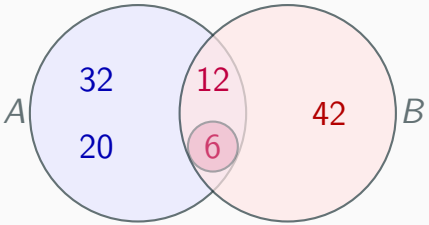
Current Step:

- **Step 1**: Initialize `result = a`. `result` $= \{6, 12, 20, 32\}$.

Set Union Trace: Step-by-Step

Union of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:

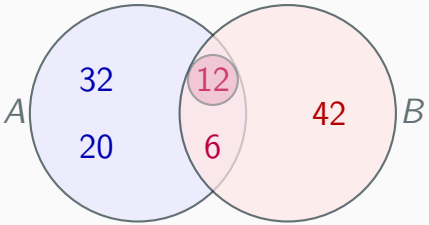


- **Step 2**: Consider first element of B: **6**.
  Check if $6 \in$ `result`. Yes!
  Insertion ignored.

Set Union Trace: Step-by-Step

Union of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:

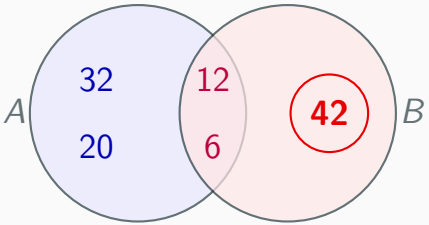


- **Step 3**: Consider second element of B: **12**.
  Check if $12 \in$ `result`. Yes!
  Insertion ignored.

Set Union Trace: Step-by-Step

Union of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:

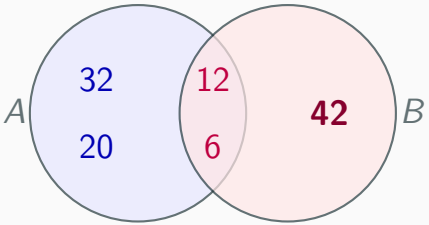


- **Step 4**: Consider third element of B: **42**.
  Check if 42 $\in$ `result`. No!
  Insert 42 into `result`.

Set Union Trace: Step-by-Step

Union of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:



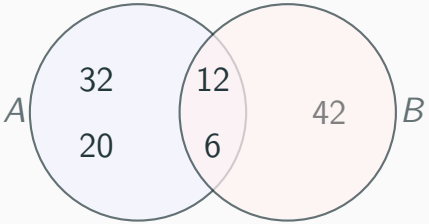
- **Step 5**: Loop ends.
  Return `result` $= \{6, 12, 20, 32, 42\}$.

Set Intersection Trace: Step-by-Step

Intersection of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:

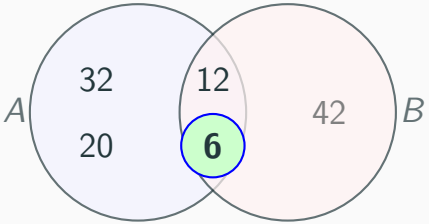
- **Step 1**: Initialize `result = {}`.



Set Intersection Trace: Step-by-Step

Intersection of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:

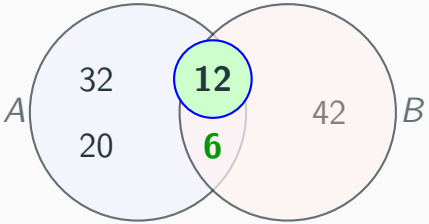


- **Step 2**: Consider $6 \in A$.
  Is 6 in B? Yes.
  Insert 6 into `result` $= \{6\}$.

Set Intersection Trace: Step-by-Step

Intersection of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:

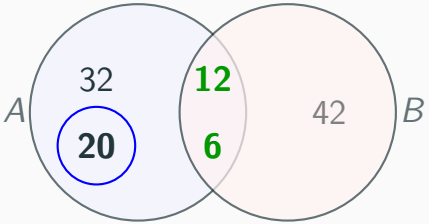


- **Step 3**: Consider $12 \in A$.
  Is 12 in B? Yes.
  Insert 12 into `result` $= \{6, 12\}$.

Set Intersection Trace: Step-by-Step

Intersection of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:

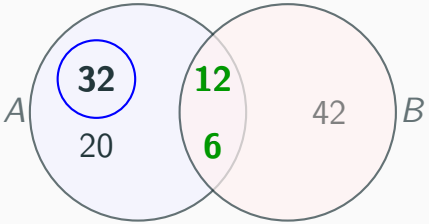


- **Step 4**: Consider $20 \in A$.
  Is 20 in B? No.
  Do not insert.

Set Intersection Trace: Step-by-Step

Intersection of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:

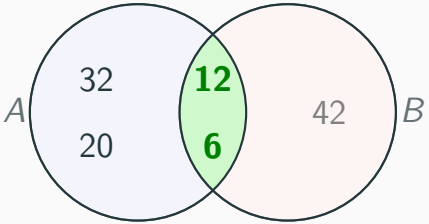


- **Step 5**: Consider $32 \in A$.
  Is 32 in B? No.
  Do not insert.

Set Intersection Trace: Step-by-Step

Intersection of $A = \{6, 12, 20, 32\}$ and $B = \{6, 12, 42\}$.

Current Step:



- **Step 6**: Loop ends.
  Return `result` $= \{6, 12\}$.

Implementing Set Difference & Set Implementations

```
1 template <typename T>
2 set<T> getDifference(set<T> a, set<T> b) {
3     set<T> result = a;
4     for (typename set<T>::iterator it = b.
          begin(); it != b.end(); ++it) {
5         result.erase(*it);
6     }
7     return result;
8 }
```

Underlying Implementations:

- **Hash-Based (`std::unordered_set`)**:
  - Average search/insert: $\mathcal{O}(1)$
  - Elements printed in arbitrary order.
- **Tree-Based (`std::set`)**:
  - Search/insert: $\mathcal{O}(\log N)$
  - Elements always printed in sorted order.

High-Level Collection Operations

Instead of basic loops, we can model Java-style high-level collection methods:

```
template <typename T>
void addAll(set<T> b, set<T>& a) {
    a.insert(b.begin(), b.end());
}

template <typename T>
bool containAll(set<T> b, set<T> a) {
    for (auto it = b.begin(); it != b.end(); ++it)
        if (a.find(*it) == a.end()) return false;
    return true;
}
```

High-Level Collection Operations

Instead of basic loops, we can model Java-style high-level collection methods:

```
template <typename T>
void removeAll(set<T> b, set<T>& a) {
    for (auto it = b.begin(); it != b.end(); ++it) a.erase(*it);
}

template <typename T>
void retainAll(set<T> b, set<T>& a) {
    for (typename set<T>::iterator it = a.begin(); it != a.end(); ) {
        if (b.find(*it) == b.end()) {
            it = a.erase(it); // Erase returns iterator to next element
        } else {
            ++it;
        }
    }
}
```

Union & Intersection via High-Level Methods

Using these primitives, we can express Set Union and Intersection very cleanly:

```
template <typename T>
set<T> getUnionUsingHelpers(set<T> setA, set<T> setB) {
    set<T> result;
    addAll(setA, result);
    addAll(setB, result);
    return result;
}

template <typename T>
set<T> getIntersectionUsingHelpers(set<T> setA, set<T> setB) {
    set<T> result;
    addAll(setA, result);
    retainAll(setB, result);
    return result;
}
```

The Map ADT

The Map ADT Definition & Applications

The **Map ADT** models a searchable collection of **key-element items** (key-value pairs).

- Each item is a pair `(k, v)`.
- Keys must be unique (multiple items with the same key are not allowed).
- Used for association, lookup, and search.

Applications

- **Address Book**: Map names to physical/email addresses.
- **DNS Resolver**: Map domain names (e.g. `cs16.net`) to IP addresses (e.g. `128.148.34.101`).
- **Student Directory**: Map student IDs to student record objects.

Fundamental Map Functions

For a map M:

- `put(k, v)`: If M does not contain key k, insert pair `(k, v)`. Otherwise, replace the existing value with v. Returns iterator to the item.
- `erase(k)`: Removes the entry with key k. (Throws error if k not found).
- `erase(p)`: Removes entry referenced by iterator p.
- `find(k)`: Returns iterator to key k, or `end()` if not found.
- `size()` / `empty()`: Returns number of entries / checks if empty.
- `begin()` / `end()`: Iterators to traverse map elements.

Map Operations Trace Example

Operation	Output	Map State
`empty()`	`true`	$\emptyset$
`put(5, A)`	iterator to `(5, A)`	{(5, A)}
`put(7, B)`	iterator to `(7, B)`	{(5, A), (7, B)}
`put(2, C)`	iterator to `(2, C)`	{(5, A), (7, B), (2, C)}
`put(2, E)`	iterator to `(2, E)`	{(5, A), (7, B), (2, E)} (value overwritten)
`find(7)`	iterator to `(7, B)`	{(5, A), (7, B), (2, E)}
`find(4)`	`end()` ("null")	{(5, A), (7, B), (2, E)}
`erase(5)`	–	{(7, B), (2, E)}

Map Bracket Operator: operator[]

The C++ `std::map` overloads the array-indexing operator:

```
ValueType & operator[] (const KeyType & key);
```

Semantics of `operator[]`

- **Key Exists**: Returns a reference to the corresponding value.
- **Key Does Not Exist**: Inserts the key with a **default value** (e.g. `0.0` or default-constructed object) into the map, and returns a reference to that new value.

The "Accidental Insertion" Side Effect

```
map<string, double> salaries;
salaries["Pat"] = 75000.00; // Inserts "Pat" with 0.0, then assigns 75000.00
cout << salaries["Jan"] << endl; // Prints 0.0, but ALSO inserts ("Jan", 0.0)
    permanently!
```

Map Lookup: operator[] vs find

Const Map Constraint

Because `operator[]` can mutate the map, it **cannot** be called on a const map reference (e.g., inside a function receiving `const map<K,V>&`).

Safe Lookup with `find`

To check/retrieve values without modifying the map, use `find()`:

```
map<string, double>::const_iterator itr = salaries.find("Chris");
if (itr == salaries.end()) {
    cout << "Not found!" << endl; // Does NOT insert "Chris"
} else {
    cout << "Salary: " << itr->second << endl;
}
```

Map Bracket Operator Trace: Step-by-Step

Trace of map operations and the resulting internal map state:

Current Operation:

- **Initial state**: Empty map.

Internal Map State:

salaries (Map)



Map Bracket Operator Trace: Step-by-Step

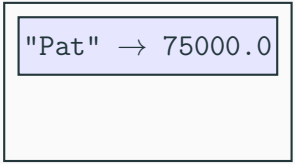
Trace of map operations and the resulting internal map state:

Current Operation:

- `salaries["Pat"] = 75000.0;`
  - Look up "Pat" → Not found.
  - Insert `("Pat", 0.0)`.
  - Assign value → `75000.0`.

Internal Map State:

salaries (Map)



Map Bracket Operator Trace: Step-by-Step

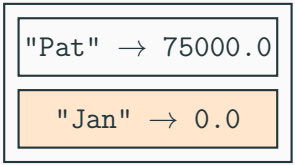
Trace of map operations and the resulting internal map state:

Current Operation:

- `cout << salaries["Jan"] << endl;`
  - Look up "Jan" → Not found.
  - Insert `("Jan", 0.0)` permanently.
  - Returns reference to `0.0`.

Internal Map State:

salaries (Map)



Map Bracket Operator Trace: Step-by-Step

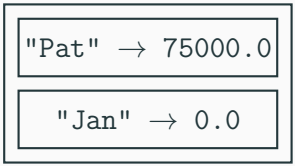
Trace of map operations and the resulting internal map state:

Current Operation:

- `itr = salaries.find("Chris");`
  - Look up "Chris" → Not found.
  - Returns `salaries.end()`.
  - **No side-effect** (no insertion!).

Internal Map State:

salaries (Map)



`"Chris"` **not inserted!**

Map Implementation: Direct Address Table

A **Direct Address Table** is a map implemented using a raw array $T[0 \ldots N-1]$.

- Keys must be integers in the range $[0, N-1]$.
- Item with key k is stored directly in $T[k]$.

Complexity

- `insert`, `find`, `erase`: $\mathcal{O}(1)$ time.
- Space: $\mathcal{O}(N)$ (size of key range).

Space Inefficiency

Direct addressing is highly space-inefficient unless the key density is high (i.e. the number of items stored n is close to the key range size N). If $n \ll N$, most array slots are wasted.

C++ STL Map: Insert and Erase Modes

```cpp
#include <iostream>
#include <map>
using namespace std;
int main() {
    map<char, int> marks; // Insertion modes
    marks.insert(pair<char, int>('b', 30)); // 1. By std::pair
    marks.insert(map<char, int>::value_type('d', 50)); // 2. By value_type
    marks['e'] = 31; // 3. By bracket operator
    // Deversion modes
    auto it = marks.find('b');
    if (it != marks.end()) {
        marks.erase(it); // 1. Erase by iterator
    }
    marks.erase('e'); // 2. Erase by key
    auto range_it = marks.find('d');// Erase range [iterator, end]
    if (range_it != marks.end()) {
        marks.erase(range_it, marks.end());
    }
}
```

C++ Map: Traversal & Emptying Loop

To print and remove elements from a map until it is empty:

```cpp
#include <iostream>
#include <map>
using namespace std;
int main() {
    map<char, int> marks = {{'a', 10}, {'b', 20}, {'c', 30}};
    cout << "Initial Size: " << marks.size() << "\n";
    while (!marks.empty()) {
        // Access key (first) and value (second) of first element
        cout << marks.begin()->first << " => " << marks.begin()->second << "\n
            ";
        marks.erase(marks.begin()); // Remove first element
    }
    cout << "Final Size: " << marks.size() << "\n";
}
```

The Dictionary ADT

The Dictionary ADT Definition

The **Dictionary ADT** is a searchable collection of key-element items.

Key Difference from Map
Duplicate keys are allowed. Multiple items can have the exact same key.

Dictionary Functions:

- `find(k)`: Returns the position/iterator of an item with key k.
- `findAll(k)`: Returns an iterator/range of all items with key k.
- `insert(k, v)`: Inserts pair.
- `erase(k)`: Erases one item with key k.

Applications:

- A language dictionary where a single word has multiple definition entries.
- Log file records grouped by timestamp or IP address.

Dictionary Operations Trace Example

Operation	Output	Dictionary State
`empty()`	`true`	$\emptyset$
`insert(5, A)`	iterator to `(5, A)`	{(5, A)}
`insert(7, B)`	iterator to `(7, B)`	{(5, A), (7, B)}
`insert(2, C)`	iterator to `(2, C)`	{(5, A), (7, B), (2, C)}
`insert(8, D)`	iterator to `(8, D)`	{(5, A), (7, B), (2, C), (8, D)}
`insert(2, E)`	iterator to `(2, E)`	{(5, A), (7, B), (2, C), (8, D), (2, E)} (no overwrite!)
`find(2)`	iterator to `(2, C)`	{(5, A), (7, B), (2, C), (8, D), (2, E)}
`findAll(2)`	range of `(2, C)` and `(2, E)`	{(5, A), (7, B), (2, C), (8, D), (2, E)}
`erase(5)`	–	{(7, B), (2, C), (8, D), (2, E)}

Unordered Implementation: Log File

A dictionary/map can be implemented as a **Log File** using an unsorted sequence (doubly-linked list or circular array).

- Elements are appended in arbitrary order.
- **Performance**:
  - `insert(k, v)`: $\mathcal{O}(1)$ time (insert at head/tail).
  - `find(k)` / `erase(k)`: $\mathcal{O}(N)$ time (must scan the entire sequence).
  - Space: $\mathcal{O}(N)$ where N is the number of stored elements.
- **Best Use Cases**: Ideal for historical logs where insertions are extremely frequent but lookups/deletions are rare (e.g. system login histories).

Summary of Implementations

Implementation Comparison Table

Let N be the number of elements stored in the collection.

Data Structure	Insertion	Lookup (Find)	Space Complexity
Log File	$\mathcal{O}(1)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Direct Address Table	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(KeyRange)$ (wasted slots if sparse)
Ordered Tree Map	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
Unordered Hash Map	$\mathcal{O}(1)$ (avg)	$\mathcal{O}(1)$ (avg)	$\mathcal{O}(N)$

Summary Guideline

Choose **Log File** for high-frequency writes/rare reads, **Direct Address Table** for tiny integer key spaces, **Tree Map** when range queries/sorted output are needed, and **Hash Map** for general-purpose fast lookups.